Programowanie współbieżne w C++

Lista zadań nr 11

Na pracownię 14 maja 2026 i późniejsze

Zadanie 1 (7pktów). Współbieżna lista leniwa z blokadami drobnoziarnistymi.

Przypomnij sobie algorytm leniwej listy z blokadami drobnoziarnistymi (klasa `LazyList` z rozdziału 9.7 książki "The Art of Multiprocessor Programming", 2e — TAoMP2e), następnie zaimplementuj ją w C++ i dobrze przetestuj. Zadbaj o odpowiedni interfejs funkcji `add`, `remove` oraz `contains`. W tym zadaniu zignoruj problem zwalniania pamięci dla węzłów usuwanych z listy.

Zadanie 2 (3pkty). Bezpieczne usuwanie elementów z listy leniwej I

Zaimplementuj bezpieczne usuwanie węzłów z listy z Zadania 1. Zastosuj zarządzanie pamięcią używające zliczania wątków wykonujących wrażliwe metody.

Zadanie 3 (7pktów). Bezpieczne usuwanie elementów z listy leniwej II

Zaimplementuj bezpieczne usuwanie węzłów z listy z Zadania 1. Zastosuj wskaźniki ryzyka.

Zadanie 4 (5pktów). Test współbieżnego stosu z licznikami referencji.

Uruchom i przetestuj kod współbieżnego stosu z licznikami referencji (Listing 7.11 i 7.12 z C++ Concurrency in Action 2e). Użyj address sanitizera wbudowanego w g++/clang++ lub zewnętrznego narzędzia np. valgrind. Następnie spróbuj zaburzyć mechanizm zliczania referencji tak, by wywołać błąd typu use-after-free, wykrywalny przez sanitizera.